

ARGoS-based Air Resistance — User Manual

Roi Sela

January 2026

Contents

1	Introduction	3
1.1	What you get	3
2	Repository Layout & Quick Start	3
2.1	Repository layout (high level)	3
2.2	Prerequisites (install before building/running)	3
2.3	Quick start (recommended path)	3
3	Configuration	4
3.1	Global wind	4
3.2	Wake (blocking) tuning via XML	4
3.3	Controller block and RAB notes	5
4	Which Controller Should I Use?	5
4.1	The controller you should start from	5
4.2	The other controllers are examples (inherit and extend)	5
5	How to Build a New Controller	5
5.1	Minimal ControlStep structure (recommended)	6
5.2	Important note: DriveImpulse vs SetLinearVelocity	6
5.3	Step 1: create a new controller folder	6
5.4	Step 2: add a CMake file	6
5.5	Step 3: include your controller in the examples build	6
5.6	Step 4: reference it from an experiment file	6
6	Examples (What They Teach You)	7
6.1	Blocked vs Unblocked (3 e-puck2)	7
6.2	Two Robots — No Blocking (parallel columns)	7
6.3	Two Followers Behind a Blocker	8
6.4	Three in a Row (wake chaining)	9
6.5	Foot-bot Wake Demo (multi-body)	10
6.6	Crosswind “Crab” Control (foot-bot)	10
7	Tuning Tips	11
8	Troubleshooting	11

1 Introduction

This project extends ARGoS with a simple but effective wind model and aerodynamic “blocking” (wake) between robots. It demonstrates how an upwind robot can shield a downwind robot, allowing the downwind robot to make upwind progress even when an unshielded robot cannot.

1.1 What you get

- **Global wind:** a 2D wind vector set once per experiment (`angle_deg` and `magnitude` in `cm/s`).
- **Post-step impulses:** wind and drive are applied after collision resolution, so the physics engine stays authoritative.
- **Wake / blocking:** upwind robots reduce effective wind for downwind neighbors, with smooth lateral falloff and downwind fade.
- **Qt overlay:** a wind arrow visualization in Qt-OpenGL.
- **Consistent units:** controller velocity and wind magnitude are both in `cm/s`.

2 Repository Layout & Quick Start

2.1 Repository layout (high level)

- `src/controllers/air_resistance/` — **main contributed code:** base controller `CAirResistance` (wind + wake pipeline)
- `src/loop_functions/wind_loop_functions/` — **loop functions + Qt user functions:** wind arrow overlay
- `examples/experiments/` — ready-to-run `.argos` experiments
- `examples/controllers/` — example controllers that **inherit from** `CAirResistance`
- `doc/` — Doxygen configuration + LaTeX sources (manuals)

2.2 Prerequisites (install before building/running)

Before building this repository, you must have:

- **ARGoS 3** installed **with the dynamics2d physics engine available** (Chipmunk backend). ARGoS **3.0.0-beta or newer** is recommended.
- Standard build tools: **CMake**, **make** (or Ninja), and a **C++ compiler**.
- **Robot plugins** depending on which experiments you want to run:
 - **foot-bot:** typically included with standard ARGoS distributions.
 - **e-puck2:** must be installed separately (upstream plugin). This repo does *not* ship the upstream e-puck2 robot plugin. If you want to run the e-puck2 experiments under `examples/experiments/`, install the e-puck2 plugin and make sure ARGoS can load it.

2.3 Quick start (recommended path)

From the repository root:

```
mkdir -p build
cd build
cmake ..
make -j
```

Run an example (also from the repository root):

```
argos3 -c examples/experiments/airResistance_two_with_block.argos
```

Notes

- Example `.argos` files load controller/loop-function libraries from `build/lib/...` using relative paths.
- If you want to skip building example controllers:

```
cmake -DBUILD_EXAMPLE_CONTROLLERS=OFF ..
```

3 Configuration

3.1 Global wind

Wind is configured once per experiment:

```
<configuration>
  <air_resistance angle_deg="0" magnitude="15.0"/>
</configuration>
```

3.2 Wake (blocking) tuning via XML

The wake model is tuned from the same `<air_resistance .../>` tag.

Units

- `lateral_reach_rad`, `shadow_length_rad`, `upwind_gate_rad`: **dimensionless** (multiples of the blocking robot radius)
- `gamma_boost`: **dimensionless**
- `adv_radius_min_m`, `adv_radius_max_m`: **meters**

Defaults (if omitted)

Attribute	Default
<code>lateral_reach_rad</code>	3.0
<code>shadow_length_rad</code>	4.0
<code>gamma_boost</code>	2.0
<code>upwind_gate_rad</code>	0.5
<code>adv_radius_min_m</code>	0.005
<code>adv_radius_max_m</code>	0.20

Example (copy/paste)

```
<configuration>
  <air_resistance
    angle_deg="0"
    magnitude="15.0"
    lateral_reach_rad="3.0"
    shadow_length_rad="4.0"
    gamma_boost="2.0"
    upwind_gate_rad="0.5"
  />
</configuration>
```

How the knobs affect behavior

- `shadow_length_radii` (**length**): larger $\Rightarrow$ wake extends further downwind.
- `lateral_reach_radii` (**width**): larger $\Rightarrow$ wake covers larger sideways offsets.
- `gamma_boost` (**shape**): 1.0 disables boost; larger values emphasize mid-range shielding (stronger wake core).
- `upwind_gate_radii` (**behindness gate**): prevents side-by-side neighbors from being counted as blockers.

3.3 Controller block and RAB notes

```
<controllers>
  <air_resistance_controller id="airbot"
    library="build/lib/controllers/air_resistance/libair_resistance">
    <actuators>
      <differential_steering implementation="default"/>
      <range_and_bearing implementation="default"/>
    </actuators>
    <sensors>
      <positioning implementation="default"/>
      <range_and_bearing implementation="medium" medium="rab" show_rays="true"/>
    </sensors>
    <params velocity="15.0"/>
  </air_resistance_controller>
</controllers>
```

RAB payload contract (important)

- The base class reserves **RAB byte 0** for a robot radius value (used by wake computations).
- For formation/messaging, store your data in **bytes 1...**
- If your controller writes byte 1, set `rab_data_size` ≥ 2 for every robot.

4 Which Controller Should I Use?

4.1 The controller you should start from

`air_resistance_controller` is the intended foundation. It implements the wind+wake impulse pipeline and provides extension hooks for users to build their own behaviors.

4.2 The other controllers are examples (inherit and extend)

`wind_aware_air_resistance_controller` and `formation_template_controller` are **example derived controllers**. They demonstrate how to inherit from `CAirResistance` and extend it.

- **Wind-aware example**: demonstrates *crosswind compensation* (“crabbing”) to reduce sideways drift.
- **Formation template**: demonstrates *multi-robot coordination* using RAB bytes 1.. while respecting the base wake contract (byte 0 reserved).

5 How to Build a New Controller

This section describes the minimal steps for adding your own controller on top of `CAirResistance`.

5.1 Minimal ControlStep structure (recommended)

A minimal control-step structure looks like:

```
// 1) Apply wind + reset accumulators + broadcast radius
HandleAerodynamicsPreStep();

// 2) Your logic here: read sensors / RAB, decide cmd speed etc.

// 3) Convert desired motion to impulse
DriveImpulse(cmd_speed_cm_s);

// 4) Apply after physics tick (post-step)
HandleAerodynamicsPostStep();
```

5.2 Important note: DriveImpulse vs SetLinearVelocity

If you want the robot to be driven by the air-resistance model, you should drive using `DriveImpulse()` and not by directly setting wheel linear velocity. The base controller applies wind and drive as impulses in the physics engine (post-step). If you use `SetLinearVelocity()` as your primary motion command, you are bypassing the intended impulse-based pipeline.

When is `SetLinearVelocity()` still okay? In derived controllers you may use wheel velocities **only for steering/yaw control**, while still using `DriveImpulse()` for forward propulsion.

5.3 Step 1: create a new controller folder

Create:

```
examples/controllers/my_controller/
```

Copy one of the example controllers as a starting point (recommended: `formation_template.cpp` because it already shows safe RAB usage).

5.4 Step 2: add a CMake file

Add `examples/controllers/my_controller/CMakeLists.txt` (pattern matches existing example controllers). Then register your controller using `REGISTER_CONTROLLER(...)`.

5.5 Step 3: include your controller in the examples build

Edit `examples/controllers/CMakeLists.txt` and add:

```
add_subdirectory(my_controller)
```

Rebuild:

```
cmake --build build -j
```

5.6 Step 4: reference it from an experiment file

Create `examples/experiments/my_experiment.argos` and load your library:

```
<my_controller id="c1" library="build/lib/controllers/my_controller/libmy_controller">
  ...
  <params velocity="12.0"/>
</my_controller>
```

6 Examples (What They Teach You)

Each experiment is designed to teach a small set of configuration concepts: (i) wind vs drive speed, (ii) wake alignment vs lateral offset, (iii) wake width/length tuning, (iv) how derived controllers extend the base controller.

6.1 Blocked vs Unblocked (3 e-puck2)

Teaches: geometric alignment determines shielding.

Salient experiment-file ideas

- Wind magnitude is near controller velocity to make differences obvious.
- One follower is directly downwind (max overlap); one is laterally offset (reduced overlap).

What happens if you change wake width/length?

- Increase `lateral_reach_rad`: the lateral robot can become partially blocked too.
- Increase `shadow_length_rad`: robots farther behind can still experience shielding.
- Increase both and watch as the previously stationary unblocked robot now moves.

Run

```
argos3 -c examples/experiments/airResistance_blocked_vs_unblocked.argos
```

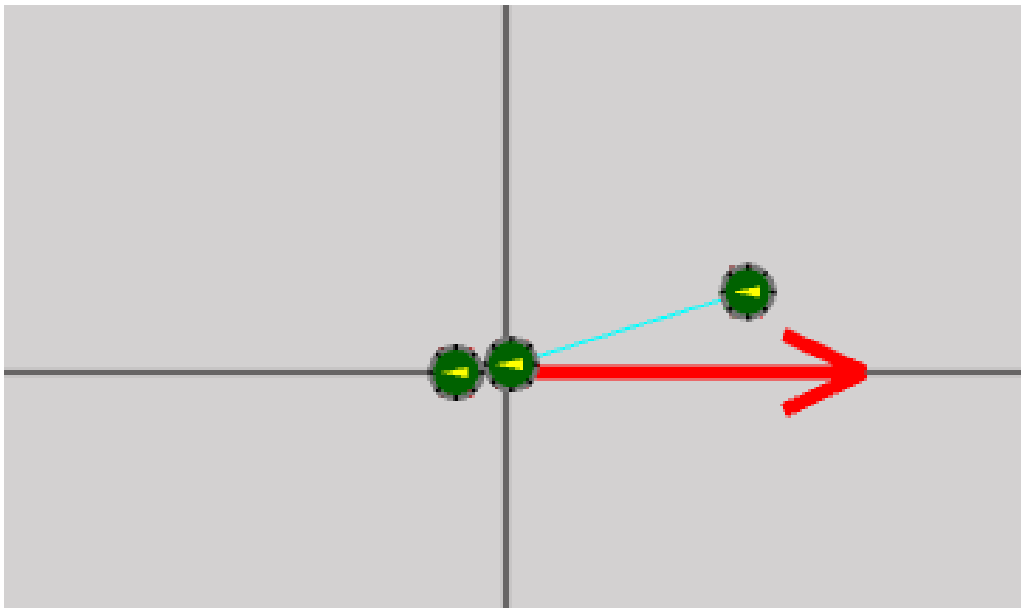


Figure 1: Blocked vs Unblocked: the robot in the wake progresses upwind; the lateral robot experiences less shielding.

6.2 Two Robots — No Blocking (parallel columns)

Teaches: lateral separation prevents wake overlap.

Salient experiment-file ideas

- Robots are separated in Y enough that wake overlap is minimal.

Run

```
argos3 -c examples/experiments/airResistance_two_no_block.argos
```

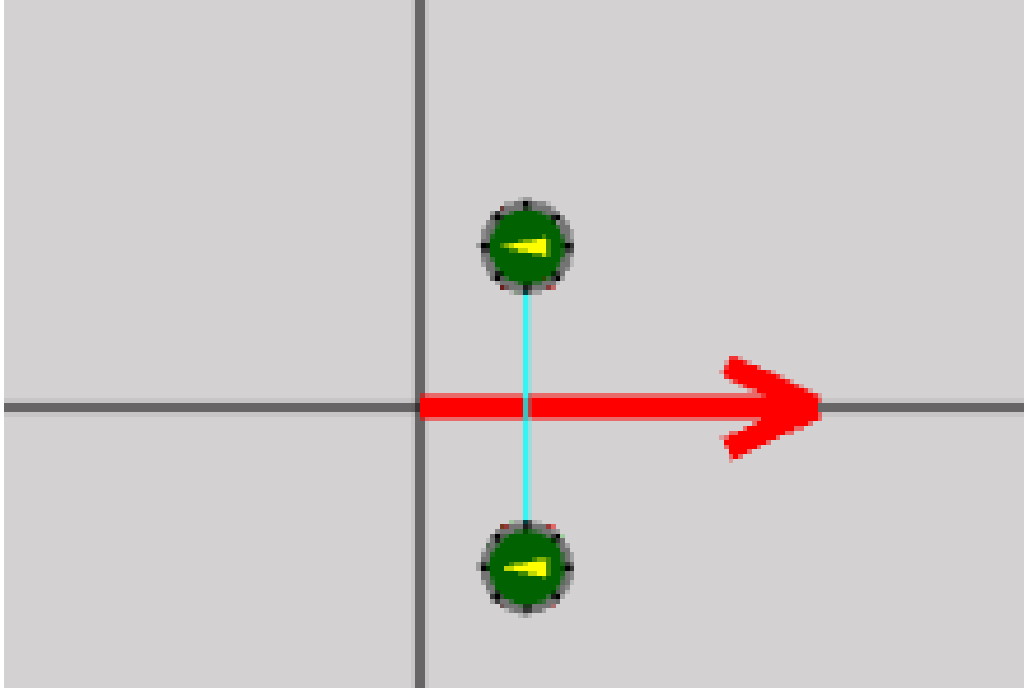


Figure 2: Two robots in parallel columns: no wake overlap for the default wake width.

6.3 Two Followers Behind a Blocker

Teaches: partial wake coverage for small lateral offsets.

Salient experiment-file ideas

- One blocker upwind; two followers behind with small Y offsets.

Tweak suggestions

- Increase `upwind_gate_radii` to make the followers stop when they are further away from the leader.
- Decrease `upwind_gate_radii` to make the followers stop when they are closer to the leader.

Run

```
argos3 -c examples/experiments/airResistance_two_with_block.argos
```

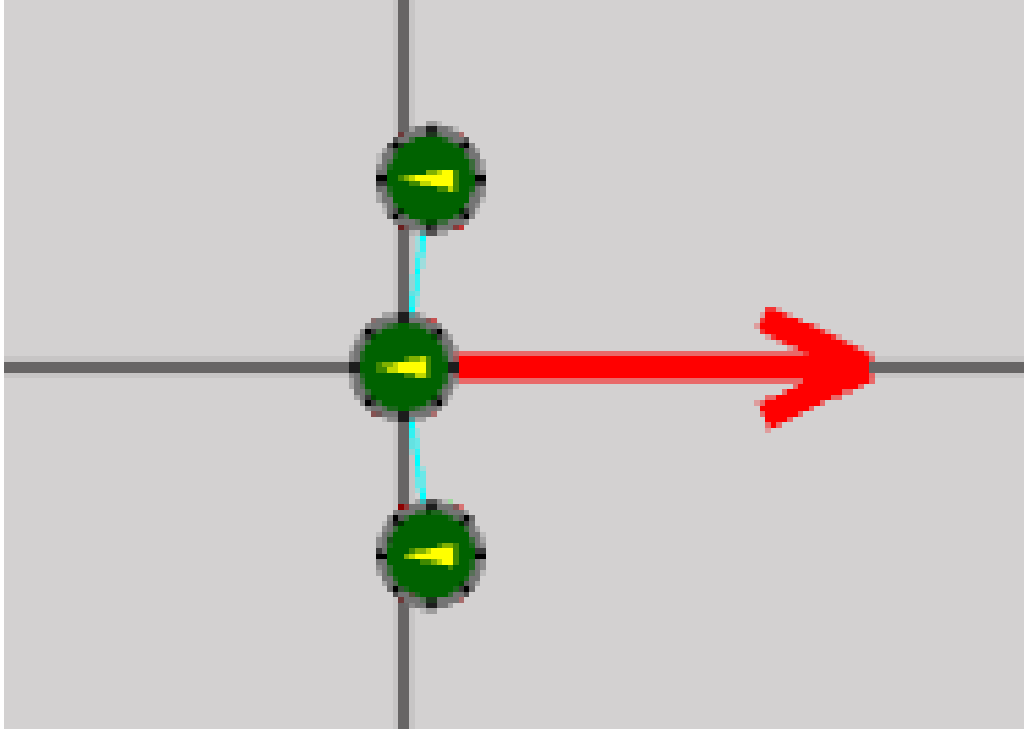



Figure 3: Two followers behind a single blocker; both can benefit if within the wake width.

6.4 Three in a Row (wake chaining)

Teaches: wake fades with downwind distance (diminishing benefit).

Salient experiment-file ideas

- Robots are placed on the wind axis with increasing downwind distance.
- The second robot gets faster as he approaches the stationary leader, the third gets slower as he loses the shielding of the second robot.

Tweak suggestions

- Increase `shadow_length_radii` to extend the wake further (third robot benefits more).
- Decrease `shadow_length_radii` to shorten the wake (third robot may lose shielding).

Run

```
argos3 -c examples/experiments/airResistance_three_in_row.argos
```

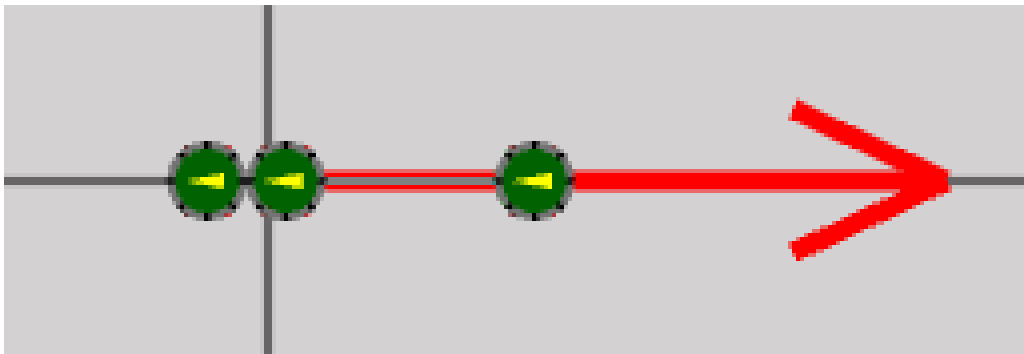


Figure 4: Wake chaining with three robots aligned downwind: the benefit decreases with distance.

6.5 Foot-bot Wake Demo (multi-body)

Teaches: the wake logic applies across robot types.

Run

```
argos3 -c examples/experiments/airResistance_foot_bot_blocking.argos
```

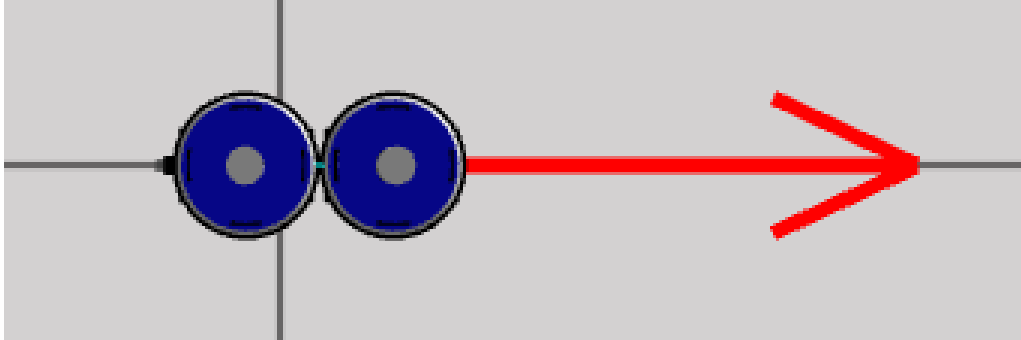


Figure 5: Wake effect with foot-bots (multi-body dynamics).

6.6 Crosswind “Crab” Control (foot-bot)

Teaches: how a derived controller can extend the base controller to compensate crosswind drift.

Salient experiment-file ideas

- Wind is perpendicular to intended motion so drift is obvious.
- Compares the base controller vs wind-aware derived controller under the same wind.

Run

```
argos3 -c examples/experiments/wind_crab_footbot.argos
```

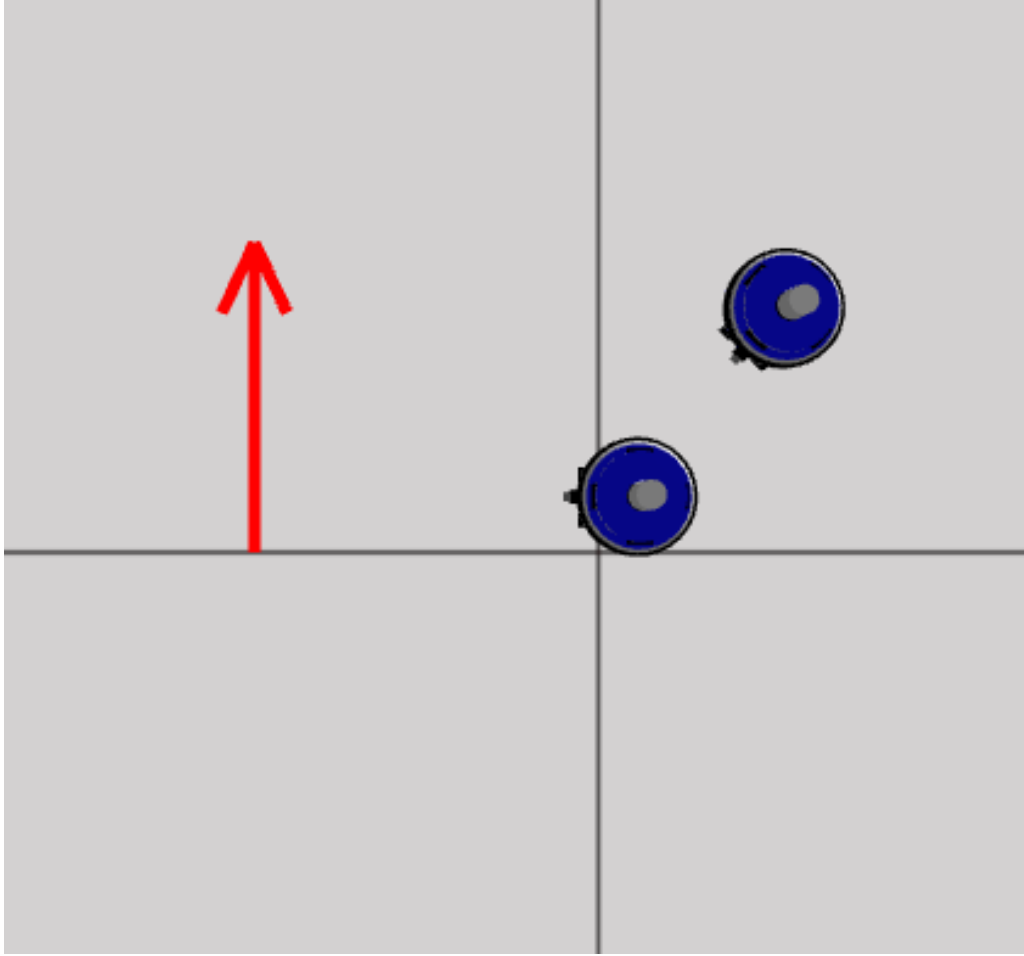


Figure 6: Crosswind compensation: the wind-aware controller reduces lateral drift by “crabbing” into the wind.

7 Tuning Tips

- Set `magnitude` $\approx$ `velocity` to make wake differences visually obvious.
- To make wake longer/shorter: tune `shadow_length_radii`.
- To make wake wider/narrower: tune `lateral_reach_radii`.
- If everything drifts downwind: increase `velocity` or reduce `magnitude`.

8 Troubleshooting

- **Plugins not found:** run from repo root so relative library paths resolve.
- **No wind arrow:** ensure Qt-OpenGL is enabled and the wind user-functions library is loaded in the `.argos`.
- **Robots not moving:** if `wind` $\approx$ `drive` and robots are unshielded, being stationary can be expected.
- **Formation/messaging bugs:** ensure `rab_data_size` is large enough if you use bytes 1.. (byte 0 is reserved).

Appendix A: Example Summary

Example	Experiment file	Robots
Blocked vs Unblocked	examples/experiments/airResistance_blocked_vs_unblocked.argos	3 × e-puck2
Two robots — no blocking	examples/experiments/airResistance_two_no_block.argos	2 × e-puck2
Two followers + blocker	examples/experiments/airResistance_two_with_block.argos	3 × e-puck2
Three in a row	examples/experiments/airResistance_three_in_row.argos	3 × e-puck2
Foot-bot wake demo	examples/experiments/airResistance_foot_bot_blocking.argos	2 × foot-bot
Crosswind crab control	examples/experiments/wind_crab_footbot.argos	2 × foot-bot